

LUCRAREA DE LABORATOR №2

STRUCTURI DINAMICE DE DATE. LISTE SIMPLU ÎNLĂNȚUITE

Scopul lucrării:

1. Studiarea operațiilor cu date dinamice.
2. Realizarea operațiilor cu structuri dinamice de date (lista liniară).
3. Studiarea mecanismului de prelucrare a elementelor structurilor dinamice de date.
4. Proiectarea algoritmilor cu structuri dinamice de date.

Indicații metodice:

1. Datele pot fi introduse de la tastatură pe unul sau mai multe rânduri.
2. Datelor de tip logic li se pot atribui valori în secțiunea definirii constantelor sau prin instrucțiunea de atribuire.
3. În regim dialog (interactiv) de afișat mesaje sugestive despre natura datelor citite de la tastatură.
4. De afișat datele de ieșire prin antet.

Structurile dinamice sunt structuri de date al căror număr de componente se modifică în timpul execuției programului. Structurile dinamice sunt mult mai flexibile decât cele statice și extrem de avantajoase acolo unde este necesară această flexibilitate.

O **listă liniară** este formată din noduri. Fiecare nod reprezintă un articol cu 2 câmpuri:

- un câmp informațional, care poate fi de orice tip;
- un câmp de legătură, care va fi de tip reper.

Câmpurile de legătură a tuturor nodurilor, în afară de ultimul nod, conțin adresele nodurilor următoare. Adresa primului nod se află într-o variabilă reper, alocată static, iar câmpul de legătură a ultimului nod conține valoarea `Nil`, astfel specificându-se sfârșitul listei.

	Varianta Pascal	Varianta C/C++
Type	<pre>Nod = Record inf: Integer; urm: ^Nod; End;</pre>	<pre>struct nod { int inf; nod *urm};</pre>
Var	<pre>Lista: ^Nod;</pre>	

Asupra listelor liniare se fac următoarele operații:

- crearea listei vide;
 - inserarea unui element în listă;
- ștergerea unui element din listă;
 - parcurgerea listei.

PROBLEME REZOLVATE:

1. Căutarea și ștergerea unui număr în listă simplu înlănțuită.

Se citește dintr-un fișier text un șir de numere separate prin spațiu cu care se creează o listă simplu înlănțuită în ordinea în care sunt citite numerele din fișier. Se mai citește de la tastatură un număr X . Să se caute și să se șteargă din listă nodul care conține acest număr.

Limbajul de programare C

```

1. #include <fstream>
2. #include <iostream>
3. using namespace std;
4. struct nod {int info;
5.             nod *urm;};
6. void adauga_nod(nod *&prim, nod *&ultim,
7.               int n)
8. {
9.     prim=new nod; ultim=prim;
10.    prim->info=n; prim->urm=NULL;
11. }
12. void adauga_ultim (nod *&ultim, int n)
13. {
14.     nod *p=new nod; p->info=n; p->urm=NULL;
15.     ultim->urm=p; ultim=p;
16. }
17. void elimina_prim (nod *&prim)
18. {
19.     nod *q=prim; prim=prim->urm; delete q;
20. }
21. void elimina_urm (nod *p)
22. {
23.     nod *q=p->urm; p->urm=p->urm->urm;
24.     delete q;
25. }
26. void afisare (nod *prim)
27. {
28.     for (nod *p=prim; p!=NULL; p=p->urm)
29.         cout<<p->info<<" "; cout<<endl;
30. }
31. int main ()
32. {
33.     int x, n;
34.     fstream fout;
35.     fout.open("lista.txt",ios::in);
36.     nod *prim, *ultim, *p;
37.     cout<<"x= "; cin>>x;
38.     if (fout)
39.     { fout>>n;
40.       adauga_nod(prim,ultim,n);
41.       while (!fout.eof())
42.       { fout>>n;
43.         adauga_ultim(ultim,n);
44.       }
45.     }
46.     fout.close();
47.     if (prim->info==x) elimina_prim(prim);
48.     else
49.     { p=cauta(prim,x);
50.       if (p->urm!=NULL) elimina_urm(p);

```

Limbajul de programare Pascal

```

Program Lista;
Type
  Nod = Record
    info: integer;
    urm: ^Nod;
  end;

Var x, n: integer;
    prim, ultim, p:^Nod;
    fout:text;

procedure adauga_nod (Var prim, ultim:^Nod;
                    n: integer);
begin
  New(prim);
  prim^.info:=n; prim^.urm:=Nil;
  ultim:=prim;
end;

procedure adauga_ultim(Var ultim:^Nod; n:integer);
Var p:^Nod;
begin
  New(p); p^.info:=n; p^.urm:=Nil;
  ultim^.urm:=p; ultim:=p;
end;

procedure cauta (Var p:^Nod; x: integer);
begin
  p:=prim;
  while (p^.urm<>Nil) and (p^.urm^.info<>x) do
    p:=p^.urm;
end;

procedure elimina_prim (Var prim:^Nod);
Var q:^Nod;
begin
  q:=prim; prim:=prim^.urm;
  dispose(q);
end;

procedure elimina_urm (p:^Nod);
Var q:^Nod;
begin
  q:=p^.urm; p^.urm:=p^.urm^.urm;
  dispose(q);
end;

procedure afisare (prim:^Nod);
Var p:^Nod;
begin
  p:=prim;
  while p<>Nil do
  begin
    write(p^.info, ' ');
    p:=p^.urm;
  end;
end;

Begin
  assign(fout, 'lista.txt'); reset(fout);
  write('Introduceti x= '); readln(x);
  if not Eof(fout) then
  begin
    readln(fout,n);

```

<pre> 56. } 57. afisare(prim); 58. } 59. 60. 61. 62. 63. 64. 65. 66. 67. 68. 69. 70. 71. 72. 73. </pre>	<pre> adauga_nod(prim,ultim,n); while not eof(fout) do begin readln(fout,n); adauga_ultim(ultim,n); end; end; close(fout); afisare(prim); if prim^.info=x then elimina_prim(prim) else begin p:=prim; cauta(p,x); if (p^.urm<>Nil) then elimina_urm(p); end; afisare(prim); End. </pre>
---	---

2. Adăugarea unui număr după elementul care posedă proprietatea dată.

Se citește dintr-un fișier text un șir de numere separate prin spațiu cu care se creează o listă simplu înlanțuită în ordinea în care sunt citite numerele din fișier. Să se verifice dacă lista conține numai numere distincte și apoi să se adauge după fiecare număr impar din listă valoare 100 și să se afișeze numerele din lista.

Limbaajul de programare C	Limbaajul de programare Pascal
<pre> 1. #include <fstream> 2. #include <iostream> 3. using namespace std; 4. struct nod {int info; 5. nod *urm;}; 6. void adauga_nod(nod *&prim, nod *&ultim, 7. int n) 8. { 9. prim=new nod; ultim=prim; 10. prim->info=n; prim->urm=NULL; 11. } 12. void adauga_ultim (nod *&ultim, int n) 13. { 14. nod *p=new nod; p->info=n; p->urm=NULL; 15. ultim->urm=p; ultim=p; 16. } 17. void adauga_dupa(nod *&q, nod *&ultim) 18. { 19. nod *p=new nod; p->info=100; 20. p->urm=q->urm; q->urm=p; 21. if (q==ultim) ultim=p; 22. } 23. int distincte (nod *prim) 24. { 25. for (nod *p=prim; p->urm!=NULL;p=p->urm) 26. for (nod *q=p->urm;q!=NULL;q=q->urm) 27. if (p->info==q->info) return 0; 28. return 1; 29. } 30. void prelucrare (nod *prim, nod *&ultim) 31. { 32. for (nod *p=prim; p!=NULL; p=p->urm) </pre>	<pre> Program Lista; Type Nod=Record info: integer; urm: ^Nod; end; Var n: integer; prim, ultim, p:^Nod; fout:text; procedure adauga_nod (Var prim, ultim:^Nod; n: integer); begin New(prim); prim^.info:=n; prim^.urm:=Nil; ultim:=prim; end; procedure adauga_ultim(Var ultim:^Nod; n:integer); Var p:^Nod; begin New(p); p^.info:=n; p^.urm:=Nil; ultim^.urm:=p; ultim:=p; end; procedure adauga_dupa(Var q,ultim: ^Nod); Var p: ^Nod; begin New(p); p^.info:=100; p^.urm:=q^.urm; q^.urm:=p; if (q=ultim) then ultim:=p; end; function distincte (prim: ^Nod):boolean; Var p,q: ^Nod; este:boolean; begin </pre>

<pre> 32. if (p->info%2==1) adauga_dupa(p,ultim); 33. } 34. void afisare (nod *prim) 35. { 36. for (nod *p=prim; p!=NULL; p=p->urm) 37. cout<<p->info<<" "; cout<<endl; 38. } 39. int main () 40. { 41. int n; 42. fstream fout; 43. fout.open("lista.txt",ios::in); 44. nod *prim, *ultim, *p; 45. if (fout) 46. { fout>>n; 47. adauga_nod(prim,ultim,n); 48. while (!fout.eof()) 49. { fout>>n; 50. adauga_ultim(ultim,n); 51. } 52. } 53. fout.close(); 54. if (distincte(prim)) 55. cout<<"Elementele sunt distincte"<<endl; 56. else 57. cout<<"Elementele nu sunt distincte"<<endl; 58. prelucrare(prim,ultim); 59. afisare(prim); 60. } 61. 62. 63. 64. 65. 66. 67. 68. 69. 70. 71. 72. 73. 74. 75. 76. 77. 78. 79. 80. 81. 82. 83. 84. 85. 86. </pre>	<pre> p:=prim; este:=true; while (p^>urm<>Nil) do begin q:=p^>urm; while (q<>Nil) do begin if (p^>info=q^>info) then este:=false; q:=q^>urm; end; p:=p^>urm; end; distincte:=este; end; procedure prelucrare(prim: ^Nod; Var ultim: ^Nod); Var p: ^Nod; begin p:=prim; while (p<>Nil) do begin if (p^>info mod 2=1) then adauga_dupa(p,ultim); p:=p^>urm; end; end; procedure afisare (prim:^Nod); Var p:^Nod; begin p:=prim; while p<>Nil do begin write(p^>info, ' '); p:=p^>urm; end; end; Begin assign(fout, 'lista.txt'); reset(fout); if not Eof(fout) then begin readln(fout,n); adauga_nod(prim,ultim,n); while not eof(fout) do begin readln(fout,n); adauga_ultim(ultim,n); end; end; close(fout); afisare(prim); if (distincte(prim)) then writeln('Elementele sunt distincte') else writeln('Elementele nu sunt distincte'); prelucrare(prim,ultim); afisare(prim); End. </pre>
--	---

Probleme propuse:

Elaborați algoritmi pentru rezolvarea următoarelor probleme:

1. Creați și afișați lista de numere reale introduse de la tastatură (până la introducerea 0).
1) Să se calculeze media aritmetică a elementelor pozitive din listă;
2) Să se afișeze numerele în ordinea inversă formării listei.
2. Creați listă liniară simplu înlănțuită **Consumator** din înregistrări ce conțin: *numele, vârsta și numărul de telefon*.
1) Să se afișeze pe ecran informația despre primii cinci consumatori;
2) Să se afișeze pe ecran consumatorii cu vârsta mai mare de 20 ani.
3. Creați listă liniară simplu înlănțuită **Student** care conține următoarea informație: *Numele studentului, Anul de studii, Grupa, Nota medie*. Să se afișeze lista studenților pe grupe.
4. Creați listă liniară simplu înlănțuită **Plan_coord** care conține coordonatele (x, y) a unor puncte din plan. Să se afișeze pe ecran punctele ce se află în interiorul cercului cu centru în punctul C(a, b) și raza R.
5. Creați două liste liniare de numere întregi introduse de la tastatură până la indicarea numărului 0. Să se verifice, dacă listele sunt identice.
6. Creați listă liniară simplu înlănțuită **Consumator** din înregistrări ce conțin informația despre *numele persoanelor și telefoanele lor*. Să se șteargă din lista creată datele consumatorului numele căruia se indică de la tastatură și să se afișeze lista obținută.
7. Creați listă liniară simplu înlănțuită **Plan_coord** care conține coordonatele (x, y) a unor puncte din plan. Să se afișeze pe ecran punctele ce formează segmente de lungime maximală.
8. Creați listă liniară simplu înlănțuită **Covor** în care se păstrează informația despre *lungimea și lățimea* unor covoare. Să se creeze o altă listă, care va conține ariile fiecărui covor și să se afișeze lista obținută.
9. Creați listă liniară simplu înlănțuită **Cărți** ce conține înregistrări cu următoarea informație: *numărul cărții, denumirea cărții, numele autorului, anul ediției, numărul de exemplare*. Să se determine, dacă în listă există cartea, denumirea căreia se introduce de la tastatură.
10. Creați listă liniară simplu înlănțuită **Dreptunghi**, care conține lungimile laturilor unor dreptunghiuri. Să se șteargă din listă dreptunghiurile cu perimetrul maximal și să se afișeze lista obținută.

11. Creați listă liniară simplu înlănțuită **Plan_coord** care conține coordonatele (x, y) a unor puncte din plan. Să se afișeze pe ecran punctele în ordinea crescătoare a coordonatei de pe axa OY.
12. Creați listă liniară simplu înlănțuită **Pasager** cu informația: *numele pasagerului, direcția și data plecării (ziua, luna, anul)*. Să se determine numărul de pasageri după data plecării.
13. Creați listă liniară simplu înlănțuită **Student** care conține următoarea informație: *Numele studentului, Anul de studii, Grupa, Nota medie*. Să se afișeze lista studenților în ordine alfabetică a numelui cu nota medie mai mare de 8.
14. Creați listă liniară simplu înlănțuită **Persoane_lista** de înregistrări cu informația: *numele, vârsta*. Să se afișeze pe ecran numele persoanei (persoanelor) cu vârsta minimală.
15. Creați listă liniară din n numere întregi introduse de la tastatură. Să se formeze a doua listă care conține elementele distincte din listă dată, împreună cu frecvența lor de apariție.
16. Creați listă liniară simplu înlănțuită **Elev**, fiecare nod conținând informații referitoare la elevii unei clase (*nume, medie, adresă, vârstă*), și apoi:
 - să se afișeze elevul cu media cea mai mică și elevul cu media cea mai mare din listă;
 - să se afișeze elevii ordonați descrescător după mediile obținute.
17. Creați listă liniară simplu înlănțuită **Elev**, fiecare nod conținând informații referitoare la elevii unei clase (*nume, medie, adresă, vârstă*), și apoi:
 - să se afișeze elevul de pe poziția k din listă;
 - să se caute informații despre un elev după numele, citit de la tastatură.
18. Creați lista **Student** care conține următoarea informație: *Numele studentului, Grupa, 3 (trei) note*. Să se creeze lista studenților în ordinea descreșterii notei medii.